

Mobile Game Analytics Dashboard

I. Project Overview

This project analyzes player behavior and game performance using event-based telemetry data collected from a mobile puzzle game.

The objective is to understand how players progress through levels, where they drop off, how they interact with boosters and advertisements, and how the game generates revenue through both advertising (Ads) and in-app purchases (IAP).

The project combines Python, PostgreSQL, SQL, and Power BI to transform raw event logs into actionable business insights that can help improve player retention, game balance, and monetization.

II. Dataset Summary

- Rows: 85 008
- Columns: 11

Column	Description
user_id	Unique player identifier
event_time_utc	Event timestamp (UTC)
app_version	Game version
country	Player country
event_name	Game event type
level	Current level
result	Win / Lose result
booster_used	Booster consumed
ad_format	Advertisement format
ad_revenue_usd	Revenue generated from ads
iap_revenue_usd	Revenue generated from purchases

- In **event_name** column have six game events:

Event:	Triggered When:
first_open	Triggered the first time a user opens the app.
level_start	Triggered when a player enters a level and starts a new attempt.
booster_use	Triggered immediately after a booster is

	successfully consumed during a level.
level_end	Triggered when a level attempt ends and the outcome (win or lose) is determined.
ad_impression	Triggered when the ad SDK returns an impression callback, confirming that the advertisement has been displayed.
iap_purchase	Triggered after the in-app purchase has been successfully verified by the server.

- Each event represents a different stage of the player's journey and enables funnel analysis, monetization analysis, and gameplay performance evaluation.

III. Data Preparation using Python

Before performing analysis, the dataset was processed using Python.

3.1. Imported CSV using pandas.

Bước 1: Load dữ liệu

```
[1]: # khai báo thư viện
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
[2]: # Đọc dữ liệu
df = pd.read_excel("DatasetOfficial.xlsx")
df.head()
```

	user_id	event_time_utc	app_version	country	event_name	level	result	booster_used	ad_format	ad_revenue_usd	iap_revenue_usd
0	U0001	2025-10-15T14:17:00Z	1.2.8	IN	first_open	0	NaN	NaN	NaN	0.0	0.0
1	U0001	2025-10-15T14:28:00Z	1.2.8	IN	level_start	1	NaN	NaN	NaN	0.0	0.0
2	U0001	2025-10-15T14:28:45Z	1.2.8	IN	level_end	1	win	NaN	NaN	0.0	0.0
3	U0001	2025-10-15T14:30:06Z	1.2.8	IN	level_start	2	NaN	NaN	NaN	0.0	0.0
4	U0001	2025-10-15T14:31:04Z	1.2.8	IN	level_end	2	lose	NaN	NaN	0.0	0.0

3.2. Checked data types using `df.info()`.

Bước 2: Kiểm tra tổng quan dữ liệu

```
[3]: # Tổng quan
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 85008 entries, 0 to 85007
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   user_id                85008 non-null  object
1   event_time_utc         85008 non-null  object
2   app_version            85008 non-null  object
3   country                85008 non-null  object
4   event_name             85008 non-null  object
5   level                  85008 non-null  int64
6   result                  34711 non-null  object
7   booster_used           5187 non-null   object
8   ad_format              9078 non-null   object
9   ad_revenue_usd         85008 non-null  float64
10  iap_revenue_usd        85008 non-null  float64
dtypes: float64(2), int64(1), object(8)
memory usage: 7.1+ MB
```

```
[4]: # Thống kê
df.describe()
```

	level	ad_revenue_usd	iap_revenue_usd
count	85008.000000	85008.000000	85008.000000
mean	21.624212	0.001397	0.022266
std	14.517516	0.004893	3.433289
min	0.000000	0.000000	0.000000
25%	10.000000	0.000000	0.000000
50%	19.000000	0.000000	0.000000
75%	32.000000	0.000000	0.000000
max	60.000000	0.039991	999.000000

Generated descriptive statistics using `df.describe()`.

[4]:
Thống kê
df.describe()

[4]:

	level	ad_revenue_usd	iap_revenue_usd
count	85008.000000	85008.000000	85008.000000
mean	21.624212	0.001397	0.022266
std	14.517516	0.004893	3.433289
min	0.000000	0.000000	0.000000
25%	10.000000	0.000000	0.000000
50%	19.000000	0.000000	0.000000
75%	32.000000	0.000000	0.000000
max	60.000000	0.039991	999.000000

- level từ 0 -> 60
- Có khách vip mua vật phẩm 999 USD, còn lại khoảng 0->75% số khách không mua gì

[5]:
Thống kê các thuộc tính đối tượng
df.describe(include="O")

[5]:

	user_id	event_time_utc	app_version	country	event_name	result	booster_used	ad_format
count	85008	85008	85008	85008	85008	34711	5187	9078
unique	1000	80503	3	5	6	2	4	2
top	U0638	2025-10-17T12:24:00Z	1.3.0	VN	level_start	win	freeze	interstitial
freq	230	5	29428	21785	34711	20536	1322	6892

3.3. Data Cleaning

Bước 3: Kiểm tra Missing Value

[6]:
df.isnull().sum()

[6]:

user_id	0
event_time_utc	0
app_version	0
country	0
event_name	0
level	0
result	50297
booster_used	79821
ad_format	75930
ad_revenue_usd	0
iap_revenue_usd	0
dtype:	int64

[7]:
df.head(5)

[7]:

	user_id	event_time_utc	app_version	country	event_name	level	result	booster_used	ad_format	ad_revenue_usd	iap_revenue_usd
0	U0001	2025-10-15T14:17:00Z	1.2.8	IN	first_open	0	NaN	NaN	NaN	0.0	0.0
1	U0001	2025-10-15T14:28:00Z	1.2.8	IN	level_start	1	NaN	NaN	NaN	0.0	0.0
2	U0001	2025-10-15T14:28:45Z	1.2.8	IN	level_end	1	win	NaN	NaN	0.0	0.0
3	U0001	2025-10-15T14:30:06Z	1.2.8	IN	level_start	2	NaN	NaN	NaN	0.0	0.0
4	U0001	2025-10-15T14:31:04Z	1.2.8	IN	level_end	2	lose	NaN	NaN	0.0	0.0

- Ta quan sát cột result, booster_used, ad_format có nhiều giá trị trống.
- Và chỉ có giá trị khi cột event_name có giá trị tương ứng.

- ví dụ: cột result chỉ có giá trị khi ở cột event_name có sự kiện level_end

Verified missing values.

Confirmed that null values in Result, Booster Used, and Ad Format were expected because these fields only exist for specific event types.

```
[8]: # Vậy khi cột event_name có sự kiện là level_end. Thì bên cột result có dòng nào trống không?
df[
    (df['event_name'] == 'level_end') &
    (
        df['result'].isna() |
        (df['result'] == '')
    )
]

[8]: user_id event_time_utc app_version country event_name level result booster_used ad_format ad_revenue_usd iap_revenue_usd

[9]: # Vậy khi cột event_name có sự kiện là ad_impression. Thì bên cột ad_format có dòng nào trống không?
df[
    (df['event_name'] == 'ad_impression') &
    (
        df['ad_format'].isna() |
        (df['ad_format'] == '')
    )
]

[9]: user_id event_time_utc app_version country event_name level result booster_used ad_format ad_revenue_usd iap_revenue_usd

[10]: # Vậy khi cột event_name có sự kiện là booster_use. Thì bên cột booster_used có dòng nào trống không?
df[
    (df['event_name'] == 'booster_use') &
    (
        df['booster_used'].isna() |
        (df['booster_used'] == '')
    )
]

[10]: user_id event_time_utc app_version country event_name level result booster_used ad_format ad_revenue_usd iap_revenue_usd
```

3.4. Checked duplicate records.

```
Bước 4: Kiểm tra Duplicate

[11]: df.duplicated().sum()
[11]: np.int64(0)

[12]: # Không có duplicate
[ ]:
[ ]:
```

3.5. Discover categorical data

Bước 5: Tìm hiểu dữ liệu categorical

```
[13]: # Trong cột phiên bản (app_version) có những giá trị nào, số lượng bao nhiêu.  
df["app_version"].value_counts()
```

```
[13]: app_version  
1.3.0    29428  
1.3.3    29388  
1.2.8    26192  
Name: count, dtype: int64
```

```
[ ]:
```

```
[14]: # Trong cột quốc gia (country) có những giá trị nào, số lượng bao nhiêu.  
df["country"].value_counts()
```

```
[14]: country  
VN    21785  
US    19684  
IN    16395  
BR    15165  
ID    11979  
Name: count, dtype: int64
```

- VN:Việt Nam
- US:Mỹ
- IN:Ấn
- BR:Brasil
- ID:In Đô

```
[ ]:
```

```
[15]: # Trong cột vật phẩm sử dụng (booster_used) có những giá trị nào, số lượng bao nhiêu.  
df["booster_used"].value_counts()
```

```
[15]: booster_used  
freeze    1322  
magnet    1313  
sizeup    1287  
compass    1265  
Name: count, dtype: int64
```

```
[15]: booster_used
      freeze      1322
      magnet      1313
      sizeup      1287
      compass      1265
      Name: count, dtype: int64
```

- freeze: đóng băng thời gian
- magnet: nam châm hút các vật phẩm về
- sizeup: tăng kích thước
- compass: chỉ đường đến vật phẩm

```
[ ]:

[16]: # Trong loại quảng cáo (ad_format) có những giá trị nào, số lượng bao nhiêu.
      df["ad_format"].value_counts()
```

```
[16]: ad_format
      interstitial      6892
      rewarded        2186
      Name: count, dtype: int64
```

- interstitial: quảng cáo chen ngang game, khi hết màn...
- rewarded: người chơi xem quảng cáo để nhận thưởng...

```
[ ]:

[17]: df["event_name"].value_counts()
```

```
[17]: event_name
      level_start      34711
      level_end      34711
      ad_impression    9078
      booster_use      5187
      first_open      1000
      iap_purchase      321
      Name: count, dtype: int64
```

- ad_impression: hiển thị quảng cáo
- iap_purchase: người chơi nạp tiền mua vật phẩm

3.6. Feature Engineering

Converted timestamps into datetime format and created additional analytical features including:

Bước 6: Feature Engineering

```
[19]: # Chuyển thời gian từ dạng string -> datetime
      df['event_time_utc'] = pd.to_datetime(df['event_time_utc'])

[ ]:

[20]: df["month"] = df["event_time_utc"].dt.month

[ ]:

[21]: df["hour"] = df["event_time_utc"].dt.hour

[ ]:

[22]: df["weekday"] = df["event_time_utc"].dt.day_name()

[ ]:

[23]: # Cả 3 phiên bản có phát hành từ ngày nào? có song song không hay gổn nhau
      version_timeline = (
          df.groupby('app_version')
              .agg(
                  first_seen=('event_time_utc', 'min'),
                  last_seen=('event_time_utc', 'max'),
                  users=('user_id', 'nunique')
              )
              .reset_index()
              .sort_values('first_seen')
      )

      version_timeline
```

	app_version	first_seen	last_seen	users
0	1.2.8	2025-10-09 08:05:00+00:00	2025-10-29 15:32:31+00:00	292
1	1.3.0	2025-10-09 08:20:00+00:00	2025-10-31 16:56:20+00:00	347
2	1.3.3	2025-10-09 08:50:00+00:00	2025-10-31 21:31:46+00:00	361

3.7. Database Integration

After cleaning, the processed dataset was loaded into PostgreSQL for SQL analysis.

```
[38]: from sqlalchemy import create_engine

      engine = create_engine(
            "postgresql+psycopg2://postgres:12345@localhost:5432/gameapp"
        )

      df.to_sql(
            "gameapp_data",
            engine,
            if_exists="replace",
            index=False
        )

[38]: 8

[ ]:

[39]: df.to_csv("Cleaned_data_gameapp.csv", index=False)
```


IV. Data Analysis using SQL

The business questions below were answered using SQL.

4.1. Which app version has the largest player base?

gameapp_data 1 X		
SELECT app_version, COUNT(DISTINCT Enter a SQL expression to filter results (use Ctrl+Space)		
Grid	A-Z app_version	123 total_users
1	1.3.3	361
2	1.3.0	347
3	1.2.8	292

4.2. Which app version generates the highest total revenue?

gameapp_data 1 X		
SELECT app_version, ROUND(SUM(to Enter a SQL expression to filter results (use Ctrl+Space)		
Grid	A-Z app_version	123 revenue
1	1.3.3	1,350.58
2	1.3.0	347.08
3	1.2.8	313.88

4.3. What is the average Lifetime Value (LTV) of players for each app version?

gameapp_data 1 X		
SELECT app_version, ROUND(SUM(tc Enter a SQL expression to filter results (use Ctrl+Space)		
Grid	A-Z app_version	123 ltv
1	1.3.3	3.74
2	1.2.8	1.07
3	1.3.0	1

4.4. What is the Day 1 (D1) retention rate for each app version?

gameapp_data 1 X				
WITH installs AS (SELECT app_versio Enter a SQL expression to filter results (use Ctrl+Space)				
Grid	A-Z app_version	123 install_users	123 retained_users	123 d1_retention
1	1.2.8	292	125	42.81
2	1.3.0	347	130	37.46
3	1.3.3	361	134	37.12

4.5. What is the Day 7 (D7) retention rate for each app version?

gameapp_data 1 X		
WITH installs AS (SELECT app_version Enter a SQL expression to filter results (use Ctrl+Space)		
Grid	A-Z app_version	123 d7_retention
1	1.2.8	26.37
2	1.3.0	19.88
3	1.3.3	19.11

4.6. What is the overall level completion (pass) rate for each app version?

gameapp_data 1 X				
SELECT app_version Close NT(*) FILTER Enter a SQL expression to filter results (use Ctrl+Space)				
Grid	A-Z app_version	123 win_attempts	123 lose_attempts	123 pass_rate_pct
1	1.2.8	5,791	4,812	54.62
2	1.3.0	7,056	4,961	58.72
3	1.3.3	7,689	4,402	63.59

4.7. What is the payer conversion rate for each app version?

gameapp_data 1 X		
SELECT app_version, ROUND(100.0 * Enter a SQL expression to filter results (use Ctrl+Space)		
Grid	A-Z app_version	123 payer_rate
1	1.2.8	25.68
2	1.3.0	23.92
3	1.3.3	22.71

4.8. What is the Average Revenue Per Paying User (ARPPU) for each app version?

gameapp_data 1 X		
SELECT app_version, ROUND(SUM(Enter a SQL expression to filter results (use Ctrl+Space)		
Grid	A-Z app_version	123 arppu
1	1.2.8	3.67
2	1.3.0	3.67
3	1.3.3	16.01

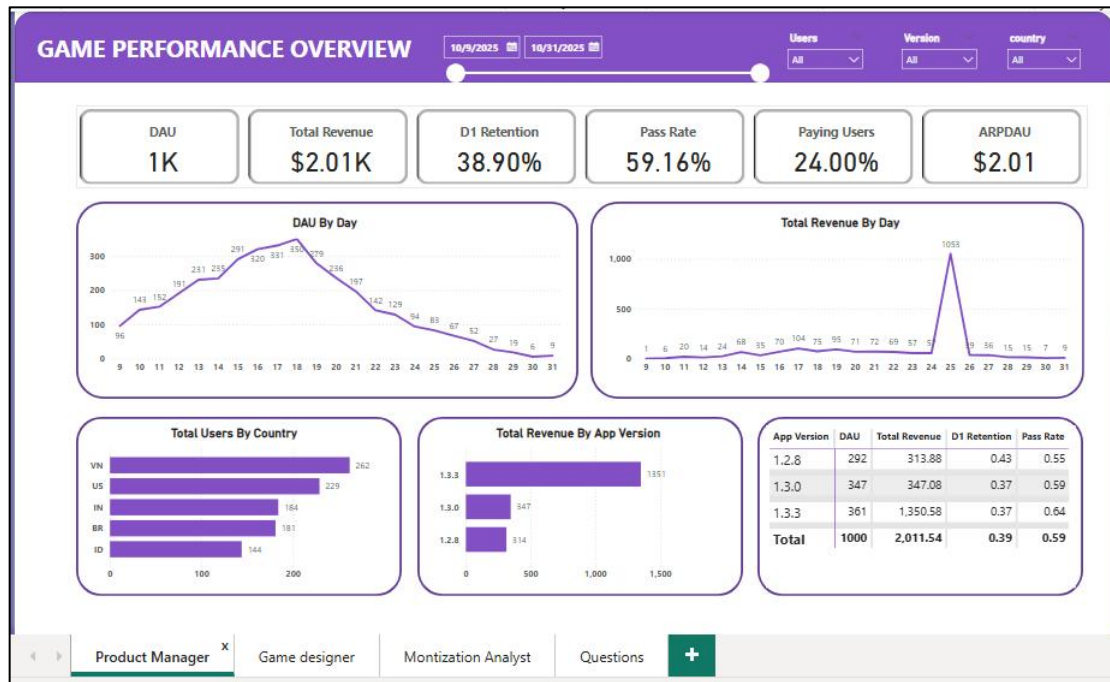
4.9. Which levels are the most difficult for players?

gameapp_data 1		gameapp_data 1 (2) X			
SELECT level, COUNT(*) FILTER (WHE		Data filter is not supported			
Grid Text Record		123 level	123 win_count	123 lose_count	123 win_rate_pct
	1	55	13	149	8.02
	2	59	9	96	8.57
	3	60	11	86	11.34
	4	56	14	109	11.38
	5	52	17	118	12.59
	6	50	27	161	14.36
	7	54	19	104	15.45
	8	58	18	96	15.79
	9	57	20	97	17.09
	10	48	39	157	19.9
	11	45	67	243	21.61

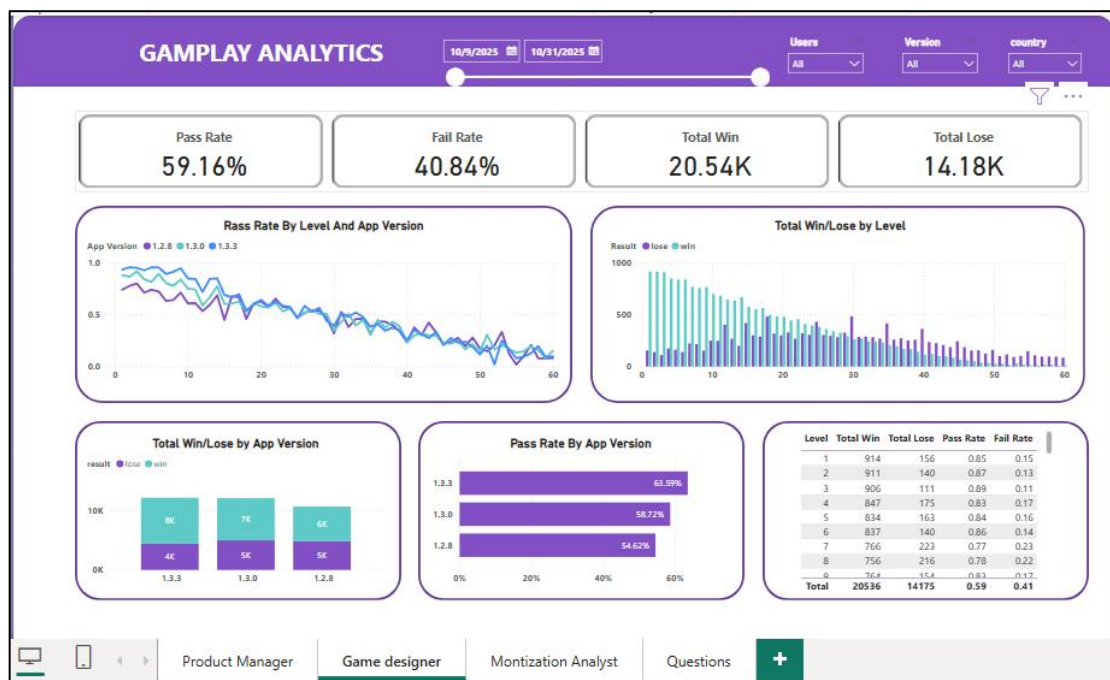
V. Power BI Dashboard

An interactive dashboard was developed in Power BI to visualize player behavior and business performance.

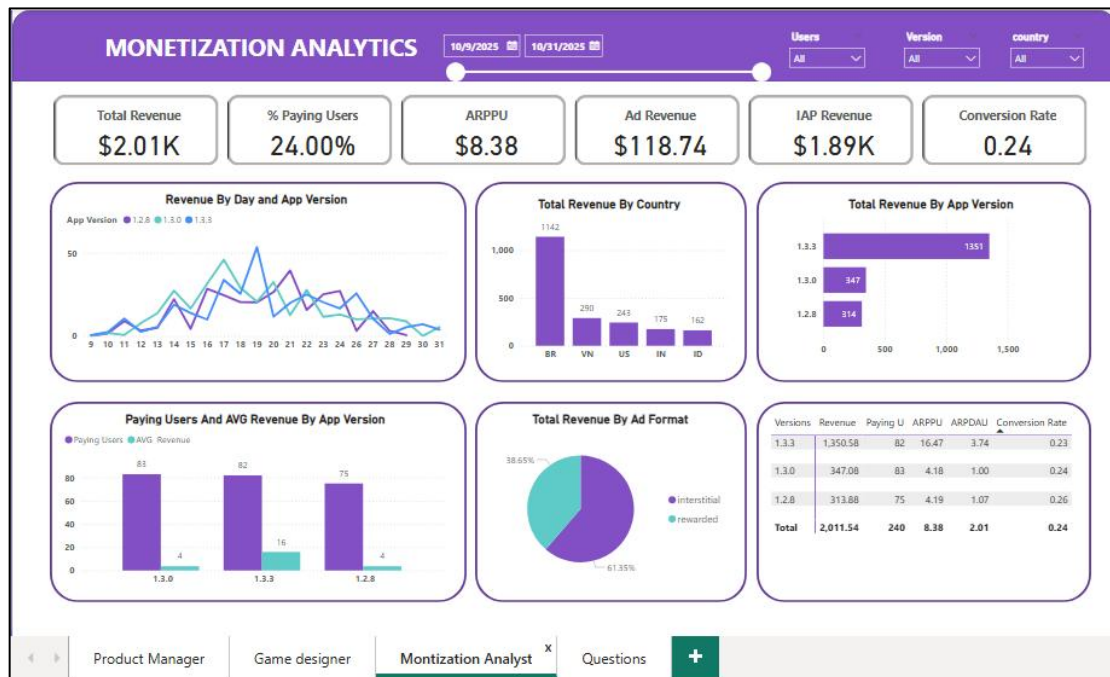
Page 1 – Executive Overview



Page 2: Gamplay analytics



Page 3: Monetization analytics



VI. Key Business Insights

6.1. Game Overview

Insight 1: Low Player Retention (D1 Retention = 38.9%)

- Only about 4 out of 10 players return to the game on the following day, while more than 60% of players churn after their first day.
- This suggests that the game's onboarding experience or early gameplay may not be engaging enough to retain new players.

Insight 2: Paying User Rate = 24%

- Approximately 24% of players made at least one purchase, which is a reasonable conversion rate.
- However, combined with an ARPPU of \$8.38, the data indicates that a relatively small group of paying users contributes a significant portion of the total revenue.

Insight 3: Revenue Is Highly Concentrated on a Single Day

- Daily revenue shows a significant spike on one particular day, indicating that revenue generation is highly concentrated rather than consistent over time.
- This suggests that the overall revenue performance may have been heavily influenced by a specific event or a high-value purchase, making the revenue trend less stable.

6.2. Gameplay Analytics

Insight 1: Overall Pass Rate = 59%

- The overall Pass Rate is 59%, indicating that the game is generally playable and most players are able to progress through the levels.

Insight 2: Pass Rate Drops Significantly in Higher Levels

- It is normal for the Pass Rate to gradually decrease as level difficulty increases. However, from Level 50 onward, the Pass Rate falls sharply to 8–20%.
- For example, Level 55 has a Pass Rate of only 8%, with 13 wins compared to 149 losses.
- This suggests that the difficulty curve becomes too steep in the late game, which may lead to player frustration and higher churn.

6.3. Monetization Analytics

Insight 1: Revenue Is Primarily Driven by In-App Purchases

- Total revenue reached \$2,011, of which In-App Purchases (IAP) contributed approximately \$1,892, while Ad Revenue accounted for only \$119.
- This indicates that over 94% of total revenue comes from IAP, with advertising contributing only about 6%. The game's monetization strategy therefore relies heavily on in-app purchases.

Insight 2: Brazil Generated the Highest Revenue

- Brazil contributed the highest total revenue among all countries.
- This presents an opportunity to further optimize monetization in the Brazilian market by evaluating pricing strategies, localized promotional offers, and region-specific in-game events to increase player engagement and revenue.

7. Business Recommendations

- Improve the first-day onboarding experience to increase D1 Retention.
- Review the difficulty balancing for levels 45 and above, with particular attention to Levels 50–60.
- For levels with low player retention, consider increasing the available completion time, providing free boosters, or adding clearer gameplay guidance and hints.
- Analyze paying user behavior using cohort analysis to better understand spending patterns and long-term value.
- Introduce more mid-priced IAP packages to improve the conversion rate.
- Version 1.3.3 remains the best-performing release even after excluding the VIP player's \$999 purchase, indicating that the improvement was not solely driven by a single high-value transaction.